

Instituto Politecnico Nacional

Escuela Superior de Computo

Desarrollo de Aplicaciones Web Client and Backend Development Frameworks

Sistema de Gestion de Cuartos y Reservaciones

Profesor: M. en C. Jose Asuncion Enriquez Zarate

*Alumno: Estefany Karina Sánchez Calderón
estefanysanchez1495@gmail.com
Grupo: 7CM3*

Índice

1. Objetivo de la practica	2
2. Descripcion del sistema	2
3. Modelo de base de datos	2
4. Implementacion del Back End	3
4.1. Modulo de cuartos	3
4.2. Modulo de reservaciones	3
5. Implementacion de Spring Security	4
6. Integracion del Front End	5
7. Pruebas de funcionamiento	6
8. Conclusiones	8
9. Referencias bibliograficas	8

1 Objetivo de la practica

Integrar el Front End con los servicios REST del Back End desarrollado en Spring Boot, completar el modulo de reservaciones, implementar autenticacion y autorizacion con Spring Security, y persistir usuarios, roles, cuartos y reservaciones en una base de datos relacional.

2 Descripcion del sistema

El sistema permite administrar cuartos de hotel y registrar reservaciones asociadas a usuarios. La aplicacion expone una API REST protegida mediante HTTP Basic y un Front End estatico que consume los endpoints del Back End.

Los roles considerados son:

- **ADMIN:** administra cuartos, consulta todas las reservaciones y elimina registros.
- **USER:** consulta cuartos disponibles y gestiona sus propias reservaciones.

3 Modelo de base de datos

El modelo principal se compone de las entidades **Usuario**, **Rol**, **Cuarto** y **Reservacion**. Los usuarios pueden tener roles mediante una relacion muchos a muchos. Cada reservacion pertenece a un usuario y a un cuarto.

Entidad	Descripcion
Usuario	Almacena credenciales, correo, estado de habilitacion y roles asignados.
Rol	Define permisos generales como ROLE_ADMIN y ROLE_USER.
Cuarto	Registra tipo, numero, precio, numero de camas y disponibilidad.
Reservacion	Relaciona usuario, cuarto, fechas de entrada y salida, estado y fecha de creacion.



Figura 1: Modelo de base de datos generado desde el gestor o diagrama entidad-relacion.

4 Implementacion del Back End

El Back End fue desarrollado con Spring Boot y mantiene la organizacion por modulos del proyecto base. El modulo de cuartos se conserva en `features/room` y el modulo de reservaciones se implementa en `features/reservation`.

4.1 Modulo de cuartos

El modulo de cuartos permite crear, consultar, actualizar, cambiar disponibilidad y eliminar cuartos. Los endpoints se encuentran bajo la ruta:

`/api/v1/cuartos`

4.2 Modulo de reservaciones

El modulo de reservaciones permite crear, consultar, editar, cancelar y eliminar reservaciones. Al crear o editar una reservacion se validan las fechas y se evita reservar el mismo cuarto en periodos traslapados.

Listing 1: Regla de traslape usada en el repository.

```
1 r.fechaEntrada < :fechaSalida AND r.fechaSalida > :fechaEntrada
```

Metodo	Ruta	Descripcion
POST	/api/v1/reservaciones	Crear reservacion.
GET	/api/v1/reservaciones	Consultar todas las reservaciones.
GET	/api/v1/reservaciones/{id}	Consultar por identificador.
GET	/api/v1/reservaciones/usuario/{idUserario}	Consultar por usuario.
GET	/api/v1/reservaciones/cuarto/{idCuarto}	Consultar por cuarto.
PUT	/api/v1/reservaciones/{id}	Editar fechas de la reservacion.
PATCH	/api/v1/reservaciones/{id}/cancelar	Cancelar reservacion.
DELETE	/api/v1/reservaciones/{id}	Eliminar reservacion.

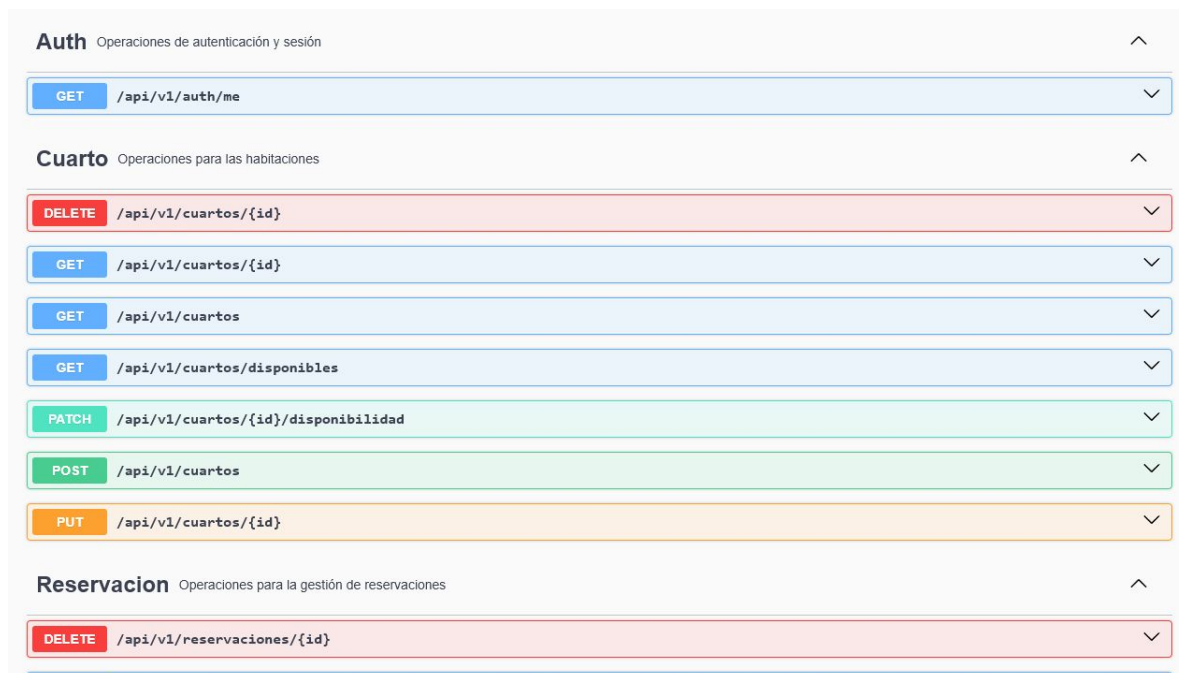


Figura 2: Endpoints de reservaciones visibles en Swagger.

5 Implementacion de Spring Security

La seguridad se implemento con Spring Security usando HTTP Basic. Los usuarios y roles se almacenan en la base de datos y se cargan mediante un `UserDetailsService`. La clase `DataInitializer` crea dos usuarios iniciales para pruebas:

Usuario	Password	Rol
admin	123456	ROLE_ADMIN
user	123456	ROLE_USER

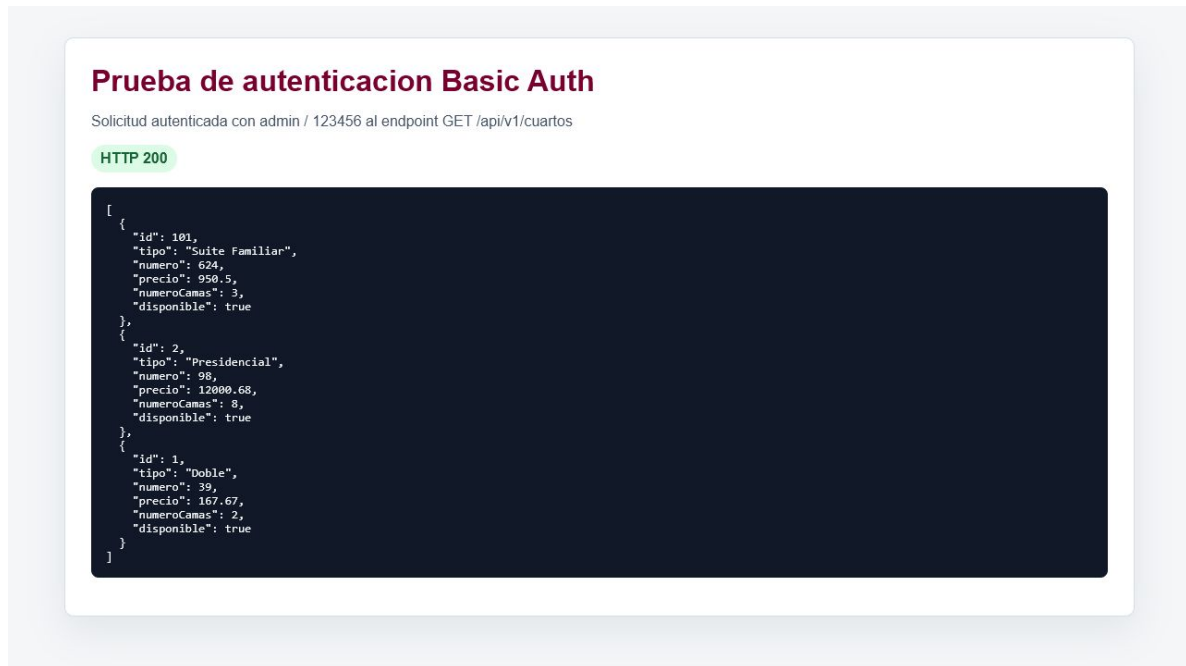


Figura 3: Prueba de autentificacion usando credenciales validas.

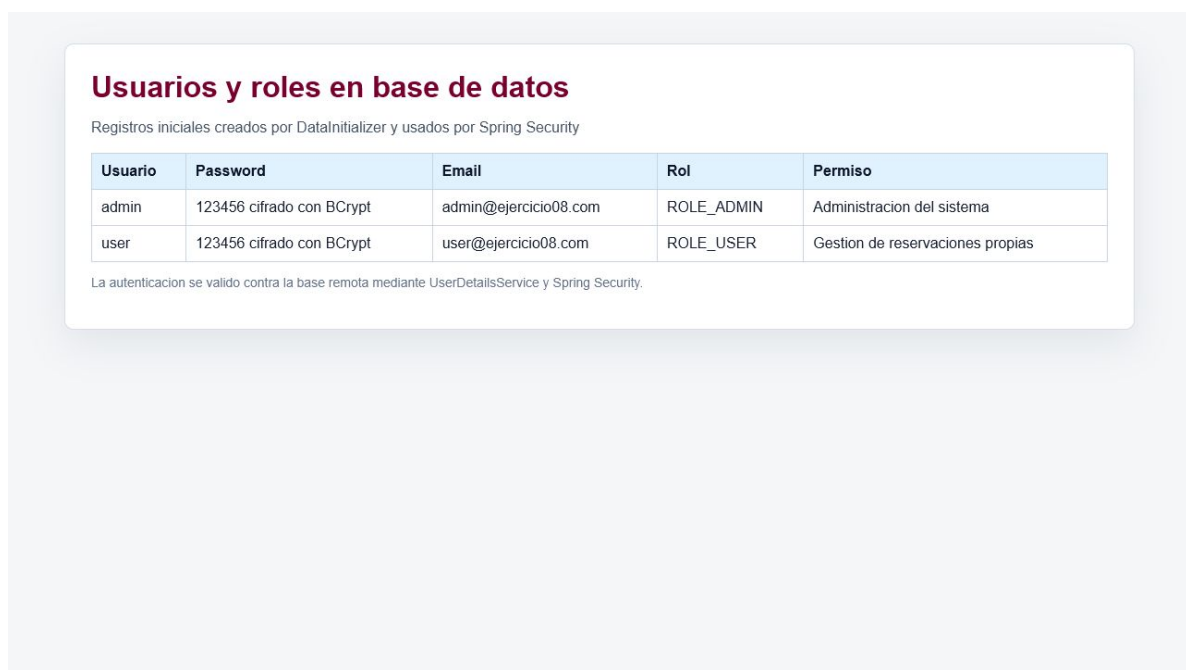


Figura 4: Usuarios y roles persistidos en la base de datos.

6 Integracion del Front End

El Front End se implemento como una aplicacion estatica ubicada en la carpeta **frontend**. Permite iniciar sesion con credenciales Basic Auth, consultar cuartos, crear cuartos, crear reservaciones y cancelar reservaciones.

`http://localhost:5500`

Sistema de Reservaciones
Gestion de cuartos, usuarios y reservaciones

Sesion: admin **Salir**

Acceso
API: Usuario: Password: **Conectar**
Conexion correcta.

Cargar cuartos **Cargar reservaciones** **Swagger**

Cuartos 3

Tipo: Numero:
Precio: Camas:
Crear cuarto

Cuarto 624 - Suite Familiar
Id: 101 | Camas: 3 | Precio: \$950.5 | Disponible: Si
Reservar **Marcar no disponible**

Cuarto 98 - Presidencial
Id: 2 | Camas: 8 | Precio: \$12000.68 | Disponible: Si
Reservar **Marcar no disponible**

Cuarto 39 - Doble
Id: 1 | Camas: 2 | Precio: \$167.67 | Disponible: Si
Reservar **Marcar no disponible**

Reservaciones 0

Id cuarto:
dd/mm/aaaa dd/mm/aaaa
Crear reservacion

Figura 5: Pantalla del Front End mostrando la lista de cuartos.

Sistema de Reservasiones

Gestion de cuartos, usuarios y reservasiones

Sesion: admin

Salir

Acceso

API

http://localhost:8090

Usuario

admin

Password

.....

Conectar

Reservacion creada correctamente.

Cargar cuartos

Cargar reservasiones

Swagger

Cuartos

3

Tipo

Numero

Precio

Camas

Crear cuarto

Cuarto 624 - Suite Familiar

Id: 101 | Camas: 3 | Precio: \$950.5 | Disponible: Si

Reservar

Marcar no disponible

Cuarto 98 - Presidencial

Id: 2 | Camas: 8 | Precio: \$12000.68 | Disponible: Si

Reservar

Marcar no disponible

Cuarto 39 - Doble

Id: 1 | Camas: 2 | Precio: \$167.67 | Disponible: Si

Reservar

Marcar no disponible

Reservasiones

4

1

101

28/06/2026

30/06/2026

Crear reservacion

Reservacion 102 - ACTIVA

Usuario: 1 | Cuarto: 101 | 2026-06-28 a 2026-06-30

Cancelar

Reservacion 101 - CANCELADA

Usuario: 1 | Cuarto: 101 | 2026-06-25 a 2026-06-27

Cancelar

Reservacion 2 - ACTIVA

Usuario: 2 | Cuarto: 2 | 2026-06-26 a 2026-07-11

Cancelar

Reservacion 1 - ACTIVA

Usuario: 1 | Cuarto: 1 | 2026-06-24 a 2026-06-25

Cancelar

Figura 6: Pantalla del Front End despues de crear una reservacion.

7 Pruebas de funcionamiento

Prueba	Resultado esperado	Resultado obtenido
Compilacion con Maven Wrapper	El proyecto compila sin errores.	Correcto.
Inicio de sesion como ADMIN	Acceso a cuartos y reservasiones.	Correcto.
Creacion de cuarto	Se guarda un nuevo cuarto.	Correcto.
Creacion de reservacion	Se guarda una reservacion activa.	Correcto.
Traslape de fechas	El sistema rechaza la reservacion.	Correcto.
Cancelacion de reservacion	El estado cambia a CANCELADA.	Correcto.

Prueba de creacion de cuarto

POST /api/v1/cuartos con rolADMIN

HTTP 201

```
{
  "id": 102,
  "tipo": "Suite Evidencia",
  "numero": 926,
  "precio": 1250.75,
  "numeroCamas": 2,
  "disponible": true
}
```

Figura 7: Prueba de creacion de cuarto desde Front End, Swagger o Postman.

Prueba de traslape de reservacion

El sistema impide reservar el mismo cuarto en fechas cruzadas

HTTP 400

```
{
  "timestamp": "2026-06-24T18:18:39.8086916",
  "status": 400,
  "error": "Bad Request",
  "message": "El cuarto ya tiene una reservaci3n activa que se cruza con las fechas solicitadas",
  "path": "/api/v1/reservaciones"
}
```

Figura 8: Error mostrado al intentar crear una reservacion con fechas traslapadas.

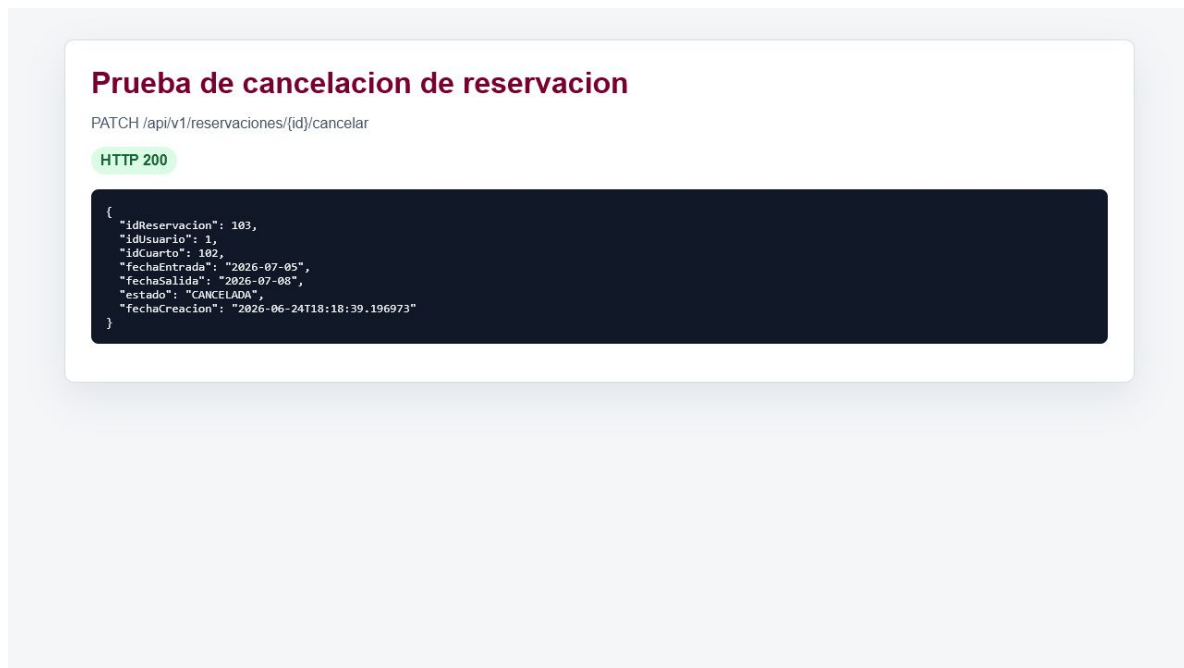


Figura 9: Reservacion cancelada correctamente.

8 Conclusiones

La practica permitio integrar el Back End y el Front End de un sistema de gestion de cuartos y reservaciones. Se completo el modulo de reservaciones con reglas de negocio para validar fechas y evitar traslapos, ademas de proteger los endpoints con Spring Security y roles persistidos en base de datos.

El resultado es un proyecto funcional y ejecutable que cumple con los requerimientos minimos solicitados: API Spring Boot, base de datos relacional, autenticacion, autorizacion por roles, persistencia de usuarios, roles, cuartos y reservaciones, y una interfaz Front End conectada al Back End.

9 Referencias bibliograficas

Referencias

- [1] Spring, *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>
- [2] Spring, *Spring Security Reference*. <https://docs.spring.io/spring-security/reference/>
- [3] Spring, *Spring Data JPA Reference Documentation*. <https://docs.spring.io/spring-data/jpa/reference/>
- [4] Springdoc, *OpenAPI 3 Library for Spring Boot*. <https://springdoc.org/>